



Università degli Studi di Salerno

Dipartimento di Informatica

Corso di Ingegneria del Software: Tecniche Avanzate

CodeSmile



Impact Analysis Document
Versione 1.0

Team

Choaib Goumri NF22500223
Mattia Gallucci NF22500127

Repository GitHub

Anno Accademico 2025–2026

Indice

1	Introduzione	1
1.1	Obiettivi del Documento	2
1.2	Change Requests	2
2	Change Request CR01	3
2.1	Analisi Statica	4
2.2	Impact Analysis	5
2.2.1	SIS (Start Impact Set)	5
2.2.2	CIS (Candidate Impact Set)	5
2.3	Implementazione CR	6
2.3.1	AIS (Actual Impact Set)	6
2.3.2	DIS (Discovered Impact Set)	6
2.3.3	FPIS (False Positive Impact Set)	6
2.4	Metriche	6
3	Change Request CR02	7
3.1	Analisi Statica	7
3.2	Impact Analysis	8
3.2.1	SIS (Start Impact Set)	8
3.2.2	CIS (Candidate Impact Set)	8
3.3	Implementazione CR	9
3.3.1	AIS (Actual Impact Set)	9
3.3.2	DIS (Discovered Impact Set)	9
3.3.3	FPIS (False Positive Impact Set)	9
3.4	Metriche	9
4	Change Request CR03	10
4.1	Impact Analysis	10
4.1.1	SIS (Start Impact Set)	11
4.1.2	CIS (Candidate Impact Set)	11
4.2	Implementazione CR	11
4.2.1	AIS (Actual Impact Set)	11
4.2.2	DIS (Discovered Impact Set)	11
4.2.3	FPIS (False Positive Impact Set)	11
4.3	Metriche	11

5	Change Request CR04	12
5.1	Analisi Statica	13
5.2	Impact Analysis	14
5.2.1	SIS (Start Impact Set)	14
5.2.2	CIS (Candidate Impact Set)	15
5.3	Implementazione CR	15
5.3.1	AIS (Actual Impact Set)	15
5.3.2	DIS (Discovered Impact Set)	15
5.3.3	FPIS (False Positive Impact Set)	15
5.4	Metriche	15
6	Change Request CR05	16
6.1	Analisi Statica	16
6.2	Impact Analysis	18
6.2.1	SIS (Start Impact Set)	18
6.2.2	CIS (Candidate Impact Set)	19
6.3	Implementazione CR	19
6.3.1	AIS (Actual Impact Set)	19
6.3.2	DIS (Discovered Impact Set)	19
6.3.3	FPIS (False Positive Impact Set)	19
6.4	Metriche	20

1 Introduzione

Questo documento ha lo scopo di analizzare formalmente l'impatto delle Change Request pianificate per il sistema Code Smile. L'obiettivo primario è guidare il processo di manutenzione evolutiva, permettendo al team di sviluppo di stimare con precisione l'estensione delle modifiche necessarie e di mitigare il rischio di regressioni o effetti collaterali indesiderati sui componenti esistenti.

La metodologia adottata si basa su un approccio white-box di analisi statica: attraverso l'esame del codice sorgente, è stata ricostruita l'architettura logica del software per mappare le interdipendenze tra moduli, servizi e classi.

Per garantire un'identificazione rigorosa delle aree critiche, l'analisi si avvale di due strumenti di modellazione principali:

1. **Grafo delle Chiamate (Call Graph):** Utilizzato per visualizzare i flussi di esecuzione e l'accoppiamento tra le entità. Questo artefatto permette di identificare i nodi orchestratori e di tracciare la propagazione delle dipendenze funzionali, facilitando la definizione degli insiemi di impatto (SIS e CIS).
2. **Matrice di Raggiungibilità (Reachability Matrix):** Uno strumento tabellare che formalizza il grado di connessione tra i componenti chiave. La matrice classifica le relazioni in base alla profondità di interazione:
 - (a) 0: Nessuna dipendenza rilevata.
 - (b) 1: Accoppiamento diretto (dipendenza immediata).
 - (c) 2+: Accoppiamento transitivo (dipendenza indiretta attraverso mediatori).

Questa modellazione strutturale è fondamentale per giustificare le previsioni di impatto e per confrontare, a posteriori, l'architettura ipotizzata con quella effettivamente implementata.

1.1 Obiettivi del Documento

Le finalità principali di questa analisi sono:

1. **Mappatura Architettuale:** Eseguire un parsing statico della codebase per delineare la gerarchia dei componenti e le relazioni di dipendenza che vincolano le modifiche.
2. **Previsione dell’Impatto:** Definire per ogni Change Request il perimetro di intervento, distinguendo tra componenti direttamente coinvolti e componenti a rischio di impatto indiretto.
3. **Giustificazione Strutturale:** Fornire evidenze oggettive (tramite grafi e matrici) che supportino la selezione dei componenti candidati alla modifica, rendendo il processo decisionale tracciabile.
4. **Valutazione dell’Accuratezza:** Misurare l’efficacia dell’analisi predittiva confrontando gli impatti stimati con le modifiche reali effettuate (AIS), utilizzando metriche standard di Information Retrieval quali Precision e Recall.

1.2 Change Requests

ID	Titolo	Descrizione
CR01	Stabilità e Accessibilità dell’Ambiente (Docker)	Automatizzazione della configurazione ambientale per rendere il codice agnostico rispetto all’esecuzione (Locale vs Container), eliminando la gestione manuale degli import e degli endpoint dei servizi.
CR02	Progress Bar CLI	Introduzione di una barra di avanzamento nella CLI per fornire feedback in tempo reale sullo stato dell’analisi, migliorando l’usabilità per progetti di grandi dimensioni.
CR03	Smell: Randomness Uncontrolled	Implementazione di una regola di analisi statica per identificare la mancata impostazione del seed (random state) negli algoritmi di Machine Learning, garantendo la riproducibilità degli esperimenti.
CR04	Smell: Hyperparameter not explicitly set	Aggiunta di una regola per rilevare l’utilizzo implicito dei valori di default per gli iperparametri critici dei modelli ML, prevenendo comportamenti imprevisti dovuti ad aggiornamenti delle librerie.
CR05	Metrica: “Densità degli Smell”	Definizione e calcolo di un indice statistico che rapporta il numero di difetti rilevati alla dimensione del codice (LOC), per prioritizzare gli interventi di refactoring.

Tabella 1.1: Tabella dei Requisiti e Miglioramenti

2 Change Request CR01

Questa richiesta di modifica nasce dall'esigenza di risolvere una criticità architetturale che compromette la manutenibilità del sistema: il forte accoppiamento tra il codice sorgente e l'infrastruttura di esecuzione.

Analisi del Contesto

Allo stato attuale, il sistema viola il principio di "Build Once, Run Anywhere". Il passaggio dall'ambiente di sviluppo locale all'ambiente containerizzato (Docker) richiede interventi manuali invasivi:

- **Gestione Import:** Gli sviluppatori devono commentare o decommentare specifiche righe di codice per adattare i percorsi dei moduli alle diverse strutture del file system.
- **Binding dei Servizi:** Gli endpoint di comunicazione nel Gateway sono definiti staticamente (hardcoded). Questo costringe a modificare il codice per passare da localhost ai nomi DNS della rete Docker interna.

Obiettivo (To-Be)

L'intervento mira a rendere l'applicazione environment-agnostic.

- **Risoluzione Dinamica:** Implementazione di logiche condizionali (try-except su import) che permettano al codice di auto-adattarsi alla struttura dei package rilevata.
- **Configurazione Esterna:** Adozione del pattern Externalized Configuration, spostando la definizione degli endpoint dal codice alle Variabili d'Ambiente, iniettate dinamicamente da Docker Compose.

2.1 Analisi Statica

L'analisi statica si è focalizzata sul sottosistema "Web Infrastructure". Il Call Graph riportato di seguito evidenzia la topologia delle comunicazioni tra i microservizi.

Dall'analisi del grafo emerge chiaramente il ruolo di Orchestrator svolto dall'API Gateway. Esso costituisce il nodo centrale della topologia a stella, dirigendo il traffico verso i tre servizi specializzati (AI Analysis, Static Analysis, Report).

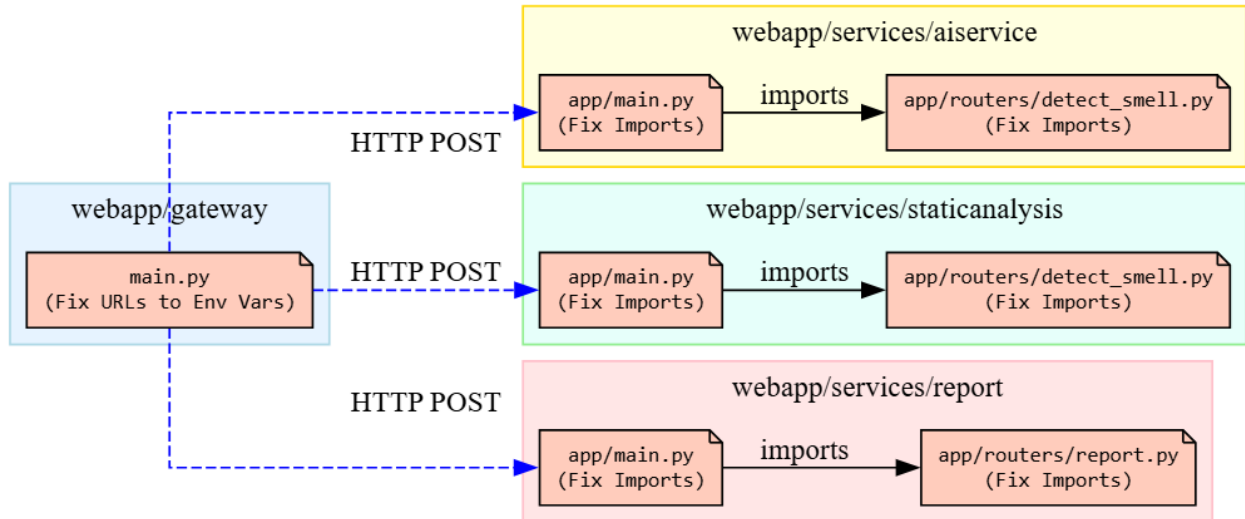


Figura 2.1: CR01 Call Graph

Matrice di raggiungibilità (Reachability Matrix)

	aiservice/main.py	aiservice/router.py	docker-compose.yml	gateway/main.py	report/main.py	report/router.py	static/main.py	static/router.py
aiservice/main.py	0	1	-	-	-	-	-	-
aiservice/router.py	-	0	-	-	-	-	-	-
docker-compose.yml	2	3	0	1	2	3	2	3
gateway/main.py	1	2	-	0	1	2	1	2
report/main.py	-	-	-	-	0	1	-	-
report/router.py	-	-	-	-	-	0	-	-
static/main.py	-	-	-	-	-	-	0	1
static/router.py	-	-	-	-	-	-	-	0

Figura 2.2: CR01 - Reachability Matrix

- **Gateway_main**: presenta valori pari a 1 nelle colonne relative agli altri servizi (*ai_main*, *static_main*, *report_main*). Questo indica una dipendenza diretta: il Gateway è l'unico componente che conosce esplicitamente l'esistenza e la posizione (endpoint) degli altri servizi. Di conseguenza, rappresenta il componente primario (*high fan-out*) su cui si concentrerà la logica di gestione dinamica degli URL tramite variabili d'ambiente.
- **(AI, Static, Report)**: presentano tutti valori pari a 0. Ciò conferma che i microservizi backend sono componenti passivi: non avviano comunicazioni verso l'esterno né verso il Gateway. Pertanto, non richiedono modifiche alla logica di routing, ma saranno impattati esclusivamente dalla normalizzazione degli import interni necessaria per l'avvio all'interno del container.

- **(Router / Utility)**: i valori incrementali (1, 2, 3) all'interno dello stesso servizio (es *ai_main* a *ai_router* fino a *ai_utility*) evidenziano la profondità della catena di chiamate interne. Le modifiche agli import dovranno quindi propagarsi lungo l'intera gerarchia, garantendo che ogni sottomodulo sia correttamente raggiungibile all'avvio dell'applicazione.

2.2 Impact Analysis

2.2.1 SIS (Start Impact Set)

Il perimetro iniziale di impatto è stato definito identificando i componenti che presentano un accoppiamento diretto con le specifiche dell'ambiente di esecuzione. Nello specifico, sono stati selezionati gli entry point dei servizi (*main.py*), critici per la gestione condizionale degli import durante il bootstrap, e i moduli di routing, che contengono le definizioni statiche (*hardcoded*) degli endpoint di comunicazione

Le classi e i file direttamente coinvolti da questa CR:

1. *webapp/gateway/main.py*
2. *webapp/services/aiservice/app/main.py*
3. *webapp/services/aiservice/app/routers/detect_smell.py*
4. *webapp/services/aiservice/app/utils/model.py*
5. *webapp/services/staticanalysis/app/main.py*
6. *webapp/services/staticanalysis/app/routers/detect_smell.py*
7. *webapp/services/staticanalysis/app/utils/static_analysis.py*
8. *webapp/services/report/app/main.py*
9. *webapp/services/report/app/routers/report.py*

2.2.2 CIS (Candidate Impact Set)

Dato l'isolamento architetturale dei microservizi confermato dalla Matrice di Raggiungibilità (i servizi backend hanno dipendenza 0 verso l'esterno), non si prevedono effetti collaterali (*ripple effects*) su componenti esterni a quelli già identificati nel SIS.

$$\text{CIS} = \text{SIS}$$

2.3 Implementazione CR

2.3.1 AIS (Actual Impact Set)

L'insieme dei file effettivamente modificati differisce parzialmente dalla previsione iniziale. Le modifiche si sono concentrate sui main.py per la gestione dell'ambiente e su specifici router e utility per la correzione dei path.

L'implementazione della CR è stata completata. I file effettivamente modificati sono:

1. webapp/gateway/main.py
2. webapp/services/aiservice/app/main.py
3. webapp/services/aiservice/app/routers/detect_smell.py
4. webapp/services/aiservice/app/utils/model.py
5. webapp/services/staticanalysis/app/main.py
6. webapp/services/staticanalysis/app/routers/detect_smell.py
7. webapp/services/staticanalysis/app/utils/static_analysis.py
8. webapp/services/report/app/main.py
9. webapp/services/report/app/routers/report.py

2.3.2 DIS (Discovered Impact Set)

Durante l'implementazione non è emersa la necessità di modificare alcun componente non previsto inizialmente.

2.3.3 FPIS (False Positive Impact Set)

Non sono stati riscontrati file inizialmente previsti che poi non hanno richiesto modifica.

2.4 Metriche

Per valutare la qualità dell'Impact Analysis, consideriamo le metriche:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{9}{9} = 1$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{9}{9} = 1$$

L'analisi è stata molto accurata, individuando quasi tutti i componenti necessari e limitando gli imprevisti a un solo file di utility inizialmente sfuggito.

3 Change Request CR02

Questa richiesta di modifica, di natura perfeettiva, mira a colmare una lacuna nell'usabilità dell'interfaccia a riga di comando (CLI) di CodeSmile, specificamente legata alla mancanza di feedback durante le operazioni a lunga esecuzione.

Analisi del Contesto

Attualmente, l'avvio di un'analisi su progetti di grandi dimensioni da terminale lascia l'utente in uno stato di incertezza. Il prompt dei comandi rimane bloccato senza emettere output fino al termine del processo, o limitandosi a stampare log rapidi e illeggibili. Questo comportamento è percepito come un difetto di reattività, portando l'utente a chiedersi se il processo sia "appeso" o ancora in corso.

Obiettivo (To-Be)

L'obiettivo è introdurre un componente visivo di Progress Bar testuale nella CLI.

- **Sincronizzazione:** La barra deve avanzare in tempo reale, sincronizzata con il progresso effettivo del motore di analisi sottostante.
- **Granularità:** Il sistema deve calcolare preventivamente il numero totale di file da analizzare (Total Workload) per scalare correttamente l'avanzamento.
- **Feedback Immediato:** Miglioramento della User (Experience (UX) tramite un indicatore deterministico (0-100%) visibile direttamente nel terminale

3.1 Analisi Statica

L'analisi statica si concentra sul componente cli e sulla sua interazione con il motore di analisi components. È necessario comprendere come il runner da terminale invochi l'analisi e come possa ricevere aggiornamenti di stato per ridisegnare la riga di output.

Il seguente Call Graph illustra le dipendenze di chiamata rilevanti per questa funzionalità.

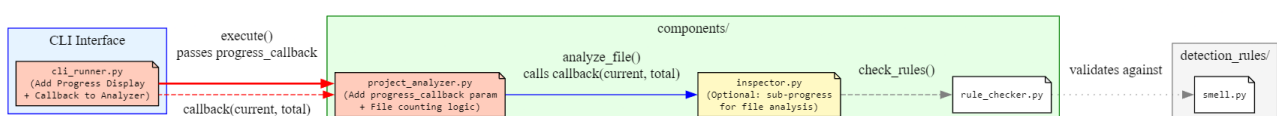


Figura 3.1: CR02 Call Graph

Il grafo illustra la catena di esecuzione avviata dal comando utente, identificando tre attori principali: la CodeSmileCLI (Caller), che funge da punto d'ingresso sincrono; il ProjectAnalyzer (Orchestrator), responsabile dell'iterazione sui file e del calcolo dell'avanzamento; e l'Inspector (Worker), che esegue l'analisi puntuale. L'analisi evidenzia una criticità architetturale nella comunicazione unidirezionale: l'orchestratore non dispone di un canale nativo per notificare il progresso al chiamante (CLI), impedendo l'aggiornamento in tempo reale dell'interfaccia.

Matrice di raggiungibilità (Reachability Matrix)

	cli/cli_runner.py	components/project_analyzer.py	components/inspector.py	components/rule_checker.py	detection_rules/smell.py
cli/cli_runner.py	0	1	2	3	4
components/project_analyzer.py	1	0	1	2	3
components/inspector.py	-	-	0	1	2
components/rule_checker.py	-	-	-	0	1
detection_rules/smell.py	-	-	-	-	0

Figura 3.2: CR02 - Reachability Matrix

- **CodeSmileCLI -> ProjectAnalyzer (1):** Dipendenza diretta. La CLI istanzia e controlla l'analizzatore. Questo è il punto in cui la Progress Bar deve essere inizializzata e gestita.
- **CodeSmileCLI -> Inspector (2+):** Dipendenza indiretta. La CLI non interagisce col singolo ispettore, quindi l'aggiornamento deve essere aggregato dal ProjectAnalyzer.
- **ProjectAnalyzer -> CodeSmileCLI (0):** Punto Critico. L'analizzatore backend ignora l'esistenza della CLI, impedendo l'aggiornamento diretto dell'interfaccia. Per rispettare il pattern MVC ed evitare dipendenze circolari, è necessario implementare un meccanismo di callback o un generatore (yield) che permetta alla CLI di consumare gli stati di avanzamento in modo asincrono

3.2 Impact Analysis

3.2.1 SIS (Start Impact Set)

L'insieme iniziale comprende il modulo di orchestrazione, responsabile del ciclo di analisi, e il file di configurazione delle dipendenze, in previsione dell'integrazione di una libreria esterna per la gestione grafica della barra.

1. components/project_analyzer.py
2. requirements.txt

3.2.2 CIS (Candidate Impact Set)

Non si prevedono impatti indiretti su altri componenti del sistema (es. Inspector o CodeSmileCLI inteso come struttura logica), poiché l'introduzione della progress bar è una modifica additiva che sfrutta le dipendenze esistenti senza alterare la logica di business dei worker.

$$\text{CIS} = \text{SIS}$$

3.3 Implementazione CR

3.3.1 AIS (Actual Impact Set)

L'insieme dei file effettivamente modificati include, come previsto, il core dell'analisi, ma si estende anche alla gestione delle dipendenze del progetto.

1. components/project_analyzer.py
2. requirements.txt

3.3.2 DIS (Discovered Impact Set)

Grazie a un'analisi preliminare accurata sulle librerie necessarie, non sono emersi componenti imprevisti durante lo sviluppo.

3.3.3 FPIS (False Positive Impact Set)

Non sono stati riscontrati file inizialmente previsti che poi non hanno richiesto modifica.

3.4 Metriche

Per valutare la qualità dell'Impact Analysis, consideriamo le metriche:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{2}{2} = 1$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{2}{2} \cong 1$$

L'analisi si è dimostrata eccellente sotto ogni aspetto. L'identificazione preventiva della necessità di aggiornare il manifesto delle dipendenze (requirements.txt) insieme al componente logico (project_analyzer.py) ha permesso di coprire l'intero perimetro della modifica, garantendo un processo di sviluppo fluido e privo di imprevisti strutturali.

4 Change Request CR03

Questa richiesta di modifica, classificata come evolutiva, riguarda l'estensione delle capacità di analisi statica del sistema CodeSmile. L'obiettivo è introdurre una nuova regola di detection per identificare problemi legati alla riproducibilità degli esperimenti di Machine Learning.

Analisi del Contesto

Questa richiesta di modifica, classificata come evolutiva, riguarda l'estensione delle capacità di analisi statica del sistema CodeSmile. L'obiettivo è introdurre una nuova regola di detection per identificare problemi legati alla riproducibilità degli esperimenti di Machine Learning.

Obiettivo (To-Be)

Implementare una nuova regola di controllo che utilizzi l'analisi dell'Abstract Syntax Tree (AST) per:

- **Identificare** le istanziazioni di modelli e funzioni sensibili alla casualità (es. `train_test_split`, `KMeans`).
- **Verificare** la presenza di parametri di controllo espliciti (es. `random_state` o `seed`).
- **Segnalare** uno smell specifico qualora tali parametri siano assenti, promuovendo la riproducibilità scientifica del codice analizzato.

4.1 Impact Analysis

Essendo la Change Request di tipo evolutivo e basata sull'architettura a plugin del sistema di regole, non sono stati previsti né riscontrati impatti sulle classi esistenti.

- Nessun componente preesistente è stato modificato o coinvolto nella logica di business o di presentazione.
- L'intervento si è limitato all'introduzione di una nuova classe dedicata, progettata per integrarsi nel framework di detection senza toccare il motore di analisi (ProjectAnalyzer) o l'interfaccia (CLI/GUI).

4.1.1 SIS (Start Impact Set)

Le classi e i file direttamente coinvolti da questa CR:

1. components/rule_checker.py
2. detection_rules/generic/randomness_uncontrolled.py (*file nuovo*)

4.1.2 CIS (Candidate Impact Set)

Non sono previsti impatti indiretti su altri componenti:

$$\text{CIS} = \text{SIS}$$

4.2 Implementazione CR

4.2.1 AIS (Actual Impact Set)

L'implementazione della CR è stata completata. I file effettivamente modificati sono:

1. components/rule_checker.py
2. detection_rules/generic/randomness_uncontrolled.py (*file nuovo*)

$$\text{AIS} = \text{SIS}$$

4.2.2 DIS (Discovered Impact Set)

Non sono stati riscontrati file non previsti in fase di analisi.

4.2.3 FPIS (False Positive Impact Set)

Tutti i file previsti hanno subito modifiche o sono stati creati come pianificato.

4.3 Metriche

Per valutare la qualità dell'Impact Analysis, consideriamo le metriche:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{2}{2} = 1$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{2}{2} = 1$$

L'analisi è risultata impeccabile, identificando con precisione sia il punto di integrazione nel motore di controllo che il nuovo modulo da implementare.

5 Change Request CR04

Questa richiesta di modifica, classificata come perfeztiva, mira a rafforzare la robustezza del codice di Machine Learning analizzato, prevenendo i rischi legati all'evoluzione delle librerie esterne.

Analisi del Contesto

Nel dominio del Machine Learning, le librerie evolvono rapidamente. Spesso gli sviluppatori istanziano modelli complessi affidandosi ai valori di default degli iperparametri, esponendosi al rischio di "Silent Failure". Se un aggiornamento della libreria modifica questi valori predefiniti, il codice continua a funzionare, ma le performance del modello possono variare drasticamente senza avvisi, compromettendo la validità degli esperimenti o la qualità del servizio.

Obiettivo (To-Be)

L'obiettivo è implementare una regola di Static Analysis che imponga l'esplicitazione degli iperparametri critici. Il sistema dovrà:

1. **Rilevare** l'istanziamento di classi di modelli noti (tramite analisi AST).
2. **Verificare** che nel costruttore siano passati esplicitamente i parametri chiave (es. `n_estimators`, `max_depth`).
3. **Segnalare** uno smell specifico se il modello è istanziato "vuoto" o privo dei parametri essenziali, obbligando lo sviluppatore a fissare la configurazione ("Pinning Configuration").

5.1 Analisi Statica

L'analisi statica si focalizza sull'integrazione della nuova regola all'interno del motore di ispezione esistente. Sebbene la regola sia un nuovo modulo, essa deve interagire con l'infrastruttura di parsing per ricevere l'AST e restituire gli smell.

Il seguente Call Graph illustra come la nuova logica di controllo si inserisce nel flusso di esecuzione dell'Inspector.

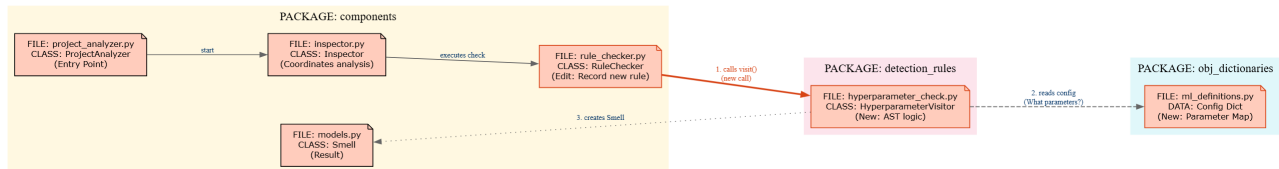


Figura 5.1: CR04 Call Graph

Il grafo evidenzia la struttura gerarchica del sistema di detection:

- **ProjectAnalyzer** : È il punto di ingresso dell'analisi statica. Avvia il processo di ispezione del progetto e delega il coordinamento all'Inspector.
- **Inspector** : Coordina l'analisi complessiva del codice. Per ogni file o costrutto rilevante, invoca il RuleChecker per applicare le regole di static analysis configurate.
- **RuleChecker** : È responsabile dell'esecuzione delle singole regole, con la CR04 viene esteso per:
 - registrare la nuova regola sugli iperparametri;
 - invocare il metodo visit() della nuova classe HyperparameterVisitor.
- **HyperparameterVisitor** (nuova logica) : Appartiene al package detection_rules ed esegue l'analisi AST:
 - intercetta le istanziazioni di modelli di Machine Learning noti;
 - verifica la presenza esplicita degli iperparametri critici nel costruttore;
 - consulta il dizionario di configurazione per sapere quali parametri controllare.
- **ModelConfig** (ml_definitions.py) : Fornisce una mappa statica dei modelli supportati e dei relativi iperparametri chiave. Questo consente di disaccoppiare la regola dalla libreria ML specifica e di gestire l'evoluzione futura delle API.
- **Smell** : Se la regola rileva un'istanziatura "vuota" o incompleta, viene creato un oggetto Smell che rappresenta il Pinning Configuration Smell, restituito poi al flusso principale dell'analisi.

La struttura conferma che l'aggiunta della CR04 sfrutta il polimorfismo: l'Inspector non deve conoscere i dettagli della nuova regola, ma deve solo poterla invocare.

Matrice di raggiungibilità (Reachability Matrix)

	HyperparameterRule	Inspector	ModelConfig	ProjectAnalyzer	RuleChecker	Smell
HyperparameterRule	0	-	1	-	-	1
Inspector	2	0	3	-	1	3
ModelConfig	-	-	0	-	-	-
ProjectAnalyzer	3	1	4	0	2	4
RuleChecker	1	-	2	-	0	2
Smell	-	-	-	-	-	0

Figura 5.2: CR04 - Reachability Matrix

- **ProjectAnalyzer** : Isolamento dell'Entry Point (Valore: 3) il ProjectAnalyzer presenta un valore di raggiungibilità pari a 3 rispetto alla HyperparameterRule. Questo dato metrico certifica che l'entry point è protetto da tre livelli di astrazione, garantendo che modifiche alla logica di detection non comportino rischi di regressione per l'avvio del sistema.
- **L'Inspector** : Mediazione dell'Inspector (Valore: 2) l'Inspector mantiene un valore di 2 verso la nuova regola. Architetturealmente, questo conferma il suo ruolo di orchestratore puro: conosce l'esistenza del meccanismo di verifica (tramite il RuleChecker), ma rimane agnostico rispetto all'implementazione specifica della CR04.
- **HyperparameterRule** : Contenimento della Foglia (Valore: 1): la nuova HyperparameterRule mostra valori di raggiungibilità in uscita pari a 1 esclusivamente verso ModelConfig e Smell. L'assenza di valori superiori a 1 o di dipendenze verso componenti di logica (come Inspector o Analyzer) definisce la regola come un componente "foglia", privo di complessità ciclonica e con impatto laterale nullo.

5.2 Impact Analysis

5.2.1 SIS (Start Impact Set)

L'insieme iniziale comprende la nuova logica di detection, il file di configurazione necessario per definire quali parametri sono obbligatori per ciascun modello, e il componente mediatore che si ipotizza debba registrare la nuova regola. Le classi e i file direttamente coinvolti da questa CR:

1. obj_dictionaries/models.csv
2. detection_rules/generic/hyperparameters_not_explicitly_set.py (*nuovo file*)
3. components/rule_checker.py

5.2.2 CIS (Candidate Impact Set)

Non sono previsti impatti indiretti su altri componenti:

$$CIS = SIS$$

5.3 Implementazione CR

5.3.1 AIS (Actual Impact Set)

L'implementazione della CR è stata completata. I file effettivamente modificati sono:

1. `obj_dictionaries/models.csv`
2. `detection_rules/generic/hyperparameters_not_explicitly_set.py`
3. `code_extractor/model_extractor.py`

5.3.2 DIS (Discovered Impact Set)

Durante l'implementazione è emersa la necessità di modificare un componente non previsto inizialmente:

1. `code_extractor/model_extractor.py`

5.3.3 FPIS (False Positive Impact Set)

L'analisi ha inizialmente ipotizzato un coinvolgimento del file che si è poi rivelato superfluo, poiché la logica esistente è risultata già adeguata senza necessità di modifiche.

1. `components/rule_checker.py`

5.4 Metriche

Per valutare la qualità dell'Impact Analysis, consideriamo le metriche:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{2}{3} \cong 0.67$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{2}{3} \cong 0.67$$

L'analisi ha mostrato un bilanciamento tra la scoperta di nuovi requisiti e la sovrastima di componenti centrali. Mentre il mancato intervento sul coordinatore delle regole indica una flessibilità del sistema superiore alle attese, la necessità di modificare l'estrattore del codice evidenzia come l'impatto funzionale si sia spostato dalla logica di controllo a quella di recupero dei dati.

6 Change Request CR05

Questa richiesta di modifica, classificata come evolutiva, propone l'introduzione di un indicatore statistico normalizzato per migliorare l'interpretabilità dei report di analisi.

Analisi del Contesto

Attualmente, il sistema riporta il numero assoluto di code smell rilevati per ogni file. Questo dato, se isolato, può essere fuorviante: un file di grandi dimensioni con pochi errori potrebbe apparire "peggiore" di un piccolo script denso di difetti solo perché il conteggio assoluto è superiore. Manca una contestualizzazione dimensionale che permetta di valutare la qualità relativa del codice.

Obiettivo (To-Be)

L'obiettivo è implementare il calcolo della Smell Density, definita come il rapporto tra il numero totale di smell rilevati e le righe di codice (LOC) del file analizzato. Il sistema dovrà:

1. **Calcolare le LOC** durante la fase di analisi.
2. **Calcolare l'indice di densità** ($Density = \frac{Smells}{LOC}$).
3. **Integrare** questo valore nei report generati, facilitando l'identificazione delle aree critiche ad alta concentrazione di difetti.

6.1 Analisi Statica

L'analisi statica si concentra sulla pipeline di elaborazione dei dati, dal momento in cui il codice viene letto fino alla generazione del report finale. È necessario identificare quali componenti devono cooperare per estrarre, trasportare e presentare la nuova metrica.

Il seguente Call Graph illustra le dipendenze funzionali tra i moduli coinvolti nell'estrazione e nel reporting :

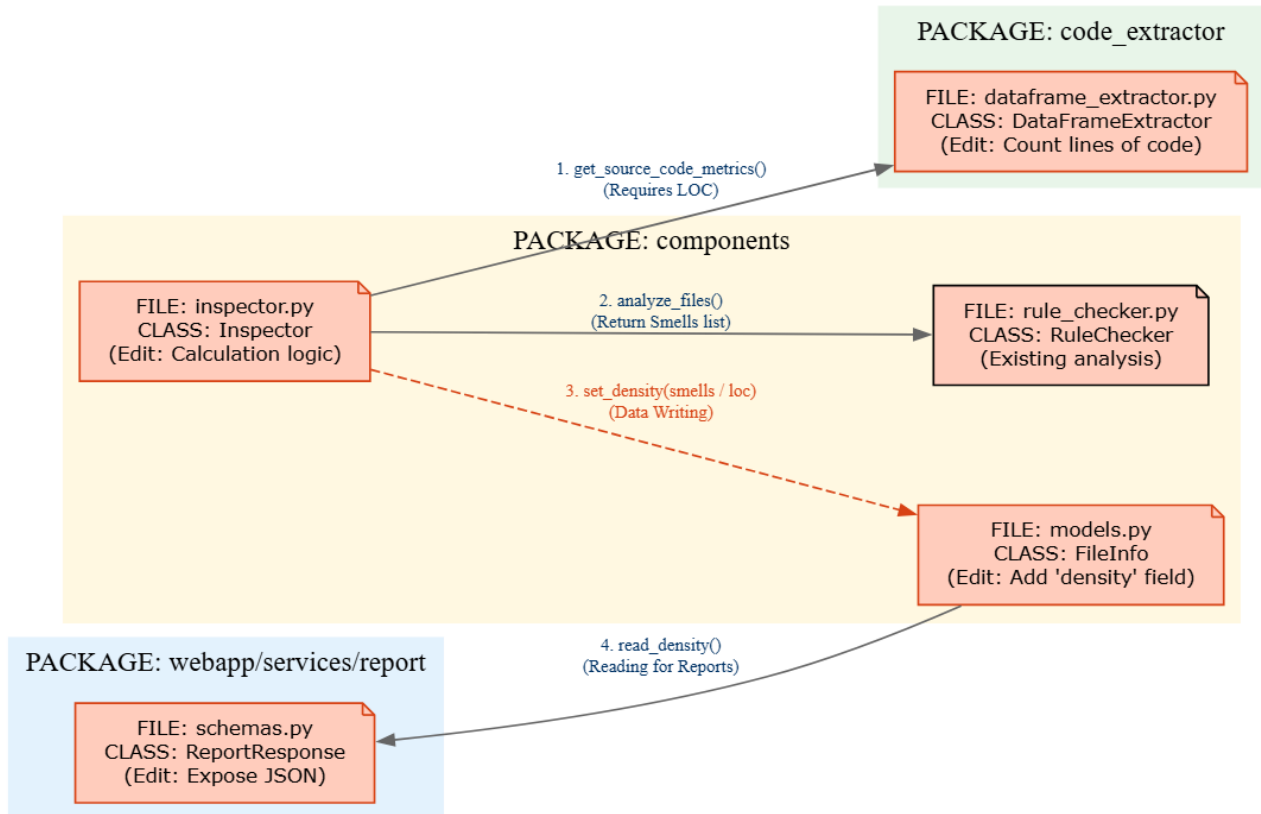


Figura 6.1: CR05 Call Graph

Il grafo evidenzia un flusso di dati lineare che coinvolge tre attori principali:

- **Inspector (Producer)**: È il componente che analizza fisicamente il file sorgente. Essendo l'unico ad avere accesso diretto al contenuto del file (AST e testo grezzo), è il candidato naturale per il calcolo delle LOC.
- **ProjectAnalyzer (Aggregator)**: Funge da orchestratore. Riceve i risultati dall'Inspector e li aggrega. Nel nuovo scenario, dovrà essere in grado di ricevere e strutturare anche il dato sulla dimensione del file.
- **ReportGenerator (Consumer)**: È il componente terminale che formatta l'output (CSV/Excel). Dipende dai dati forniti dall'analizzatore per popolare le colonne del report.

La struttura mostra chiaramente che la modifica è trasversale (Cross-Cutting): il dato "LOC" nasce nell'Inspector e deve attraversare l'intera catena fino al ReportGenerator.

Matrice di raggiungibilità (Reachability Matrix)

	DataFrameExtractor	FileInfo	GenerateReportResponse	Inspector	RuleChecker
DataFrameExtractor	0	-	-	-	-
FileInfo	-	0	-	-	-
GenerateReportResponse	-	1	0	-	-
Inspector	1	1	-	0	1
RuleChecker	-	-	-	-	0

Figura 6.2: CR05 - Reachability Matrix

- **ProjectAnalyzer -> Inspector (Valore 1):** Indica una Dipendenza Diretta di Chiamata. Il ProjectAnalyzer invoca l'Inspector. Se l'Inspector modifica la sua interfaccia di ritorno (es. restituendo un oggetto arricchito con loc), il ProjectAnalyzer ne è direttamente impattato poiché deve gestire il nuovo formato dei dati.
- **ProjectAnalyzer -> ReportGenerator (Valore 1):** Indica una Dipendenza di Flusso Dati. L'analizzatore passa i risultati aggregati al generatore di report. Poiché la CR richiede di aggiungere una colonna al report, la struttura dati passata a questo metodo dovrà cambiare, propagando l'impatto funzionale fino al layer di presentazione.
- **Altri valori (0):** Confermano l'assenza di dipendenze cicliche o inverse. L'Inspector non conosce chi usa i suoi dati (disaccoppiamento produttore-consumatore) e il ReportGenerator è un componente passivo che si limita a formattare l'input ricevuto.

6.2 Impact Analysis

6.2.1 SIS (Start Impact Set)

L'insieme iniziale comprende i componenti di frontend necessari per l'upload, le definizioni dei tipi, e i componenti core per l'orchestrazione e l'analisi. Le classi e i file direttamente coinvolti da questa CR:

1. webapp/app/upload-python/page.tsx
2. webapp/types/types.ts
3. webapp/utils/api.ts
4. components/inspector.py
5. components/project_analyzer.py
6. components/rule_checker.py
7. code_extractor/dataframe_extractor.py
8. report/report_generator.py

6.2.2 CIS (Candidate Impact Set)

Non sono previsti impatti indiretti su altri componenti:

$$\text{CIS} = \text{SIS}$$

6.3 Implementazione CR

6.3.1 AIS (Actual Impact Set)

L'implementazione della CR è stata completata. I file effettivamente modificati sono:

1. webapp/app/upload-python/page.tsx
2. webapp/types/types.ts
3. webapp/utils/api.ts
4. components/inspector.py
5. components/project_analyzer.py
6. report/report_generator.py
7. webapp/services/staticanalysis/app/routers/detect_smell.py
8. webapp/services/staticanalysis/app/schemas/responses.py
9. webapp/services/staticanalysis/app/utils/static_analysis.py

6.3.2 DIS (Discovered Impact Set)

Durante l'implementazione è emersa la necessità di modificare dei componenti non previsti inizialmente:

1. webapp/services/staticanalysis/app/routers/detect_smell.py
2. webapp/services/staticanalysis/app/schemas/responses.py
3. webapp/services/staticanalysis/app/utils/static_analysis.py

6.3.3 FPIS (False Positive Impact Set)

L'analisi preventiva ha sovrastimato l'impatto su alcuni moduli di estrazione e controllo, i quali si sono rivelati già pronti a gestire i nuovi dati senza necessità di interventi strutturali:

1. components/rule_checker.py
2. code_extractor/dataframe_extractor.py

6.4 Metriche

Per valutare la qualità dell'Impact Analysis, consideriamo le metriche:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{6}{8} = 0.75$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{6}{9} \cong 0.67$$

L'analisi ha individuato correttamente i punti di intervento sull'interfaccia e sul motore di analisi principale, ma ha mancato l'aggiornamento dei contratti API e dei router nei servizi backend.